



Formation Kubernetes

Session 8 — Terraform + K8s : Déployer un Cluster

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session



Recap Session 7

HCL, init/plan/apply, state, remote backend

10 min



Modules Terraform

Réutilisabilité, composition, syntaxe

10 min



Variables & Locals

Paramétrisation, pattern var.student_name, éviter les collisions

10 min



Pattern environments/

Staging vs Production, 1 état par env, approche RFLKT

10 min



Architectures cloud

Comparatif GKE/EKS/AKS, ressources Terraform

10 min

Recap — Session 7



HCL

Langage déclaratif Terraform — blocks, args, expressions



Init / Plan / Apply

Workflow : init (backend), plan (preview), apply (create)



State

Fichier terraform.tfstate — mappage resource ↔ infra réelle



Remote Backend

GCS / S3 — state partagé, locking, collaboration

Aujourd'hui : modules, variables, et architectures réelles à grande échelle

Modules Terraform – Réutilisabilité

Sans modules ✗

- 300+ lignes HCL pour 1 cluster
- Code dupliqué staging ↔ prod
- Difficile à maintenir et versionner
- Pas de réutilisabilité

Avec modules ✓

- Bloc de ~50 lignes réutilisable
- main.tf uniquement pour la composition
- Facile à maintenir et tester
- Réutilisable staging/prod/multi-cloud

Syntaxe d'appel

```
module "database" {  
  source = "../modules/database"  
  # Variables d'entrée  
  db_name      = var.database_name  
  db_version   = "17"  
  db_region    = var.region  
}
```

Outputs utilisés par d'autres modules

Variables, Outputs, Locals

Variables (Input)

- Paramétrisation
- `var.student_name`
- `var.region` (default)
- Validation (type, length)

Locals (Computed)

- Valeurs dérivées
- `local.app_labels`
- `local.environment_tag`
- Réutilisables dans le module

Outputs (Return)

- Exposition de données
- `output.cluster_name`
- `output.cluster_ip`
- Utilisables par d'autres modules

PATTERN CLÉ : `var.student_name` = prefix pour TOUS les noms GCP

Pattern environments/ – Staging vs Production

```
cloud-infrastructure/  
├─ environments/  
│   └─ staging/  
│       └─ main.tf          ← Modules  
composés  
│   └─ terraform.tfvars ← Variables  
staging  
│   └─ terraform.tfstate ← State staging  
│       └─ production/  
│           └─ main.tf      ← Même modules,  
vars différentes  
│       └─ terraform.tfvars ← Variables prod  
│           └─ terraform.tfstate ← State  
production  
├─ modules/  
│   └─ network/  
│   └─ cluster/  
│   └─ node_pool/  
└─ Makefile
```

Avantages

- ✓ 1 state par environnement
- ✓ Modules réutilisés (DRY)
- ✓ Variables différentes
- ✓ Isolation des erreurs
- ✓ CD pipeline distincts
- ✓ Rollback indépendant
- ✓ Chez RFLKT : staging et prod en europe-west9

Comparatif Cloud — GKE vs EKS vs AKS

Ressource	GCP (Terraform)	AWS (Terraform)	Azure (Terraform)
K8s cluster	google_container_cluster	aws_eks_cluster	azurerm_kubernetes_cluster
VPC / Network	google_compute_network	aws_vpc	azurerm_virtual_network
Subnet	google_compute_subnetwork	aws_subnet	azurerm_subnet
NAT (egress)	google_compute_router + router_nat	aws_nat_gateway	azurerm_nat_gateway
Container Registry	google_artifact_registry	aws_ecr_repository	azurerm_container_registry
IAM Workload	google_service_account	aws_iam_role (IRSA)	azurerm_user_assigned_identity

Terraform abstrait les différences — Un module Terraform GKE / EKS, mais la logique reste portable

Architecture du cluster – Ce qu'on va construire

VPC: 10.0.0.0/16

Subnet: 10.0.1.0/24 (Kubernetes)



GKE Cluster

(Control Plane géré par Google)

Node Pool

e2-small spot VMs
Autoscale: 1-3 nodes

Cloud Router + NAT (Egress Internet)

Module Walkthrough – 3 modules

network/

VPC, Subnet, Cloud Router, NAT

Outputs: vpc_id,
subnet_id, nat_ip

~60 lignes

cluster/

GKE cluster (control plane)

Outputs: cluster_name,
cluster_endpoint

~50 lignes

node_pool/

Node pool (e2-small spot, autoscale)

Outputs: node_pool_name,
node_count

~40 lignes

Chaînage : network output → cluster input → node_pool input

```
# environments/staging/main.tf
module "cluster" { vpc_id      = module.network.vpc_id }
module "node_pool" { cluster_name = module.cluster.name }
```

Pièges et coûts – IMPORTANT

terraform destroy = STOP BILLING ⚠ CRITIQUE

À la fin du TP, terraform destroy pour éviter les charges de VM qui tournent

State file = source de vérité

Ne pas supprimer / modifier à la main. Si perte : terraform import pour récupérer

Collisions de noms = terraform ERROR ⚠ CRITIQUE

Chaque var.student_name doit être UNIQUE. Sinon : 2 stagiaires = mêmes noms de ressources GCP → erreur

Quotas projet & disponibilité régionale

1 VPC + 1 Cloud Router par stagiaire : limites partagées au projet (~8 clusters en parallèle ici). Vérifier aussi la dispo régionale (GPU, machines).

Le TP – chacun crée son cluster

```
terraform/student-template/  
├─ terraform.tfvars      ← À configurer  
├─ main.tf               ← Appelle  
├─ variables.tf  
├─ outputs.tf  
└─ modules/              ← Partagés  
    ├─ network/  
    ├─ cluster/  
    └─ node_pool/
```

Timeline provisioning

1. Clone template	2 min
2. Edit terraform.tfvars	2 min
3. terraform init	1 min
4. terraform plan	2 min
5. terraform apply	8-12 min
6. Deploy app (optionnel)	5 min
7. terraform destroy	5-10 min

À la fin du TP : `terraform destroy -auto-approve` pour arrêter la facturation

Démo Live — Infrastructure RFLKT réelle

```
# cloud-infrastructure/environments/production/main.tf
module "network" {
  source = "../../modules/network"
  environment = "production"
  region = "europe-west9"
}
module "cluster" {
  source = "../../modules/cluster"
  vpc_id = module.network.vpc_id
  is_private = true      ← Cluster privé
  workload_identity = true ← IRSA pour pods
  environment = "production"
}
module "node_pool" {
  source = "../../modules/node_pool"
  cluster_name = module.cluster.name
  machine_type = "n2-standard-4"
  initial_node_count = 3
}
```

Caractéristiques RFLKT

- ✓ Private cluster : pas d'IP publique
- ✓ Workload Identity (IRSA) : pods authentifiés sans clés
- ✓ Node pool multi-tier : on-demand + spot
- ✓ Terraform state dans GCS (backend distant)

Récapitulatif

4 Concepts clés

1

Modules

Bloc réutilisable — source, variables, outputs

2

Variables + `var.student_name`

Paramétrisation, éviter collisions GCP

3

Pattern environments/

staging et production isolés, 1 state par env

4

Coûts + terraform destroy

Toujours destroy à la fin pour arrêter la facturation

Prochaine étape — Session 9 : Helm + Terraform

Déployer des applications dans le cluster avec Helm piloté par Terraform — Traefik, cert-manager et TLS pour une chaîne IaC complète de bout en bout.